Woxsen University · School of Technology

DBMS Project-Based Learning (PBL) · Final Report

Hotel Reservation and Guest Services Management System

Project 12

Submitted by: **Dandigam Ritheek**

Student ID: 25WU0102057 · Programme: AI-ML

Semester 3 · 2026

Source code, decks and prototype:
github.com/ritheekyadav01-afk/dbms-hotel-pbl-review1

Live demo: ritheekyadav01-afk.github.io/taj-ritheek

1. Abstract

Hotels depend on many linked records: rooms, rates, guests, reservations, stays, services, housekeeping and bills. Kept in separate spreadsheets, these lead to double bookings, missed charges and wrong invoices. This project designs and builds a single relational database in MySQL 8 that keeps all of them consistent, together with a working application on top of it. The schema has 12 tables normalized to third normal form (with two documented, trigger-guarded exceptions), 10 triggers and 8 CHECK constraints that enforce 8 business rules inside the database itself, and 8 reporting views. Three front ends use the same database: a Python command-line app, a Streamlit web UI, and a React and Flask web application (“Taj Ritheek”). All SQL is parameterized and every multi-step operation runs as one atomic transaction.

Contents

1 Abstract · 2 Problem, scope and objectives · 3 Requirements and actors · 4 Conceptual design (ER diagram) · 5 Relational schema and normalization · 6 Constraints, triggers and business rules · 7 SQL queries, views and reports · 8 Application (prototype) · 9 Testing and validation · 10 Results, limitations and future work · 11 How to run · 12 Conclusion

2. Problem, Scope and Objectives

2.1 Problem

A hotel manages an interconnected set of resources. When they are tracked in uncoordinated tools, rooms are double-booked, guest charges are missed or duplicated, housekeeping status is lost and invoices are wrong. The project provides one database that keeps room availability, guest stays, service billing and housekeeping consistent at all times.

2.2 Scope

In scope	Out of scope
Room types and inventory with occupancy limits; rate plans per room type; guest registration; reservations with availability search; check-in and check-out as a Stay; in-stay service requests and charges; housekeeping tasks per room and staff member; invoices and payments; seven operational reports.	Multi-property chains; staff HR, payroll and shifts; third-party OTA integration and guest self-booking portal.

2.3 Objectives

- Model the domain as an ER diagram and convert it to a relational schema in 3NF with PK, FK, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
- Enforce core business rules in the database: no overlapping bookings, valid dates, room-capacity control, non-negative amounts, no overpayment.
- Build an application for availability search, reservation, check-in and check-out, service posting, housekeeping, billing and payment.
- Produce reports using joins, nested queries, aggregation and views.

3. Requirements and Actors

Actor	Interacts with
Front desk / receptionist	Availability search, reservation, stay (check-in and check-out), invoice
Housekeeping staff / supervisor	Housekeeping tasks (assign, update status)
Service staff	Service requests and charges
Billing / accounts	Invoice, payment
Manager / admin	Room types, rate plans, reports
Guest	Source of guest and reservation data, captured by the front desk

ID	Functional requirement	Delivered by
FR1	Search room availability by date range and room type	Availability search / Book a room
FR2	Create, modify or cancel a reservation	Reservation operations
FR3	Check in (creates a Stay) and check out (closes it)	Check-in / check-out
FR4	Log a service request and generate its charge	Guest services
FR5	Assign and track housekeeping tasks	Housekeeping board
FR6	Generate an invoice (room plus service charges)	Check-out and Billing
FR7	Record payments against an invoice	Billing
FR8	Reports: occupancy, vacancy, cancellations, housekeeping, guest history, service revenue, total revenue	8 views and Reports page
FR9	Validate data (overlaps, dates, non-negative amounts)	Triggers and CHECKs

4. Conceptual Design: ER Diagram

The model groups its 12 entities into four areas: inventory and pricing (room_type, room, rate_plan), guest activity (guest, reservation, stay), in-stay services and operations (service_request, service_charge, housekeeping_task, staff) and billing (invoice, payment). A reservation exists before arrival and may be cancelled, so the actual stay is a separate entity linked 1 to 0..1.

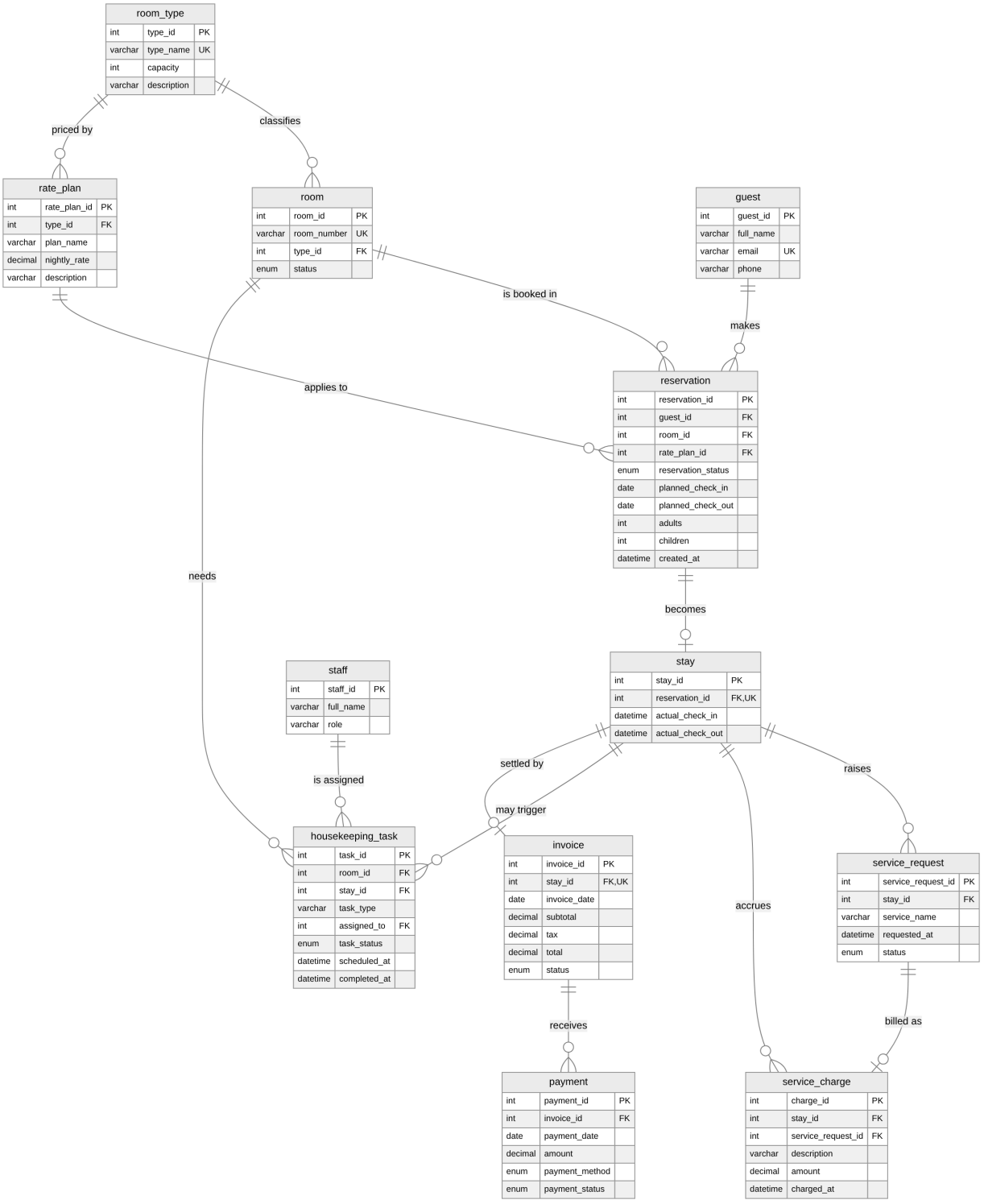


Figure 1. Entity-relationship diagram of the implemented schema (crow's foot notation).

5. Relational Schema and Normalization

5.1 Schema at a glance

12 tables · 14 foreign keys · 5 UNIQUE constraints · 8 CHECK constraints · 7 ENUM columns · 3 secondary indexes · 10 triggers · 8 views (counts read from the live MySQL schema).

5.2 Anomalies of a flat design

- **Update anomaly:** changing a room type's capacity would mean editing every booking row of that type.
- **Insert anomaly:** a new guest or room type could not be stored until a booking exists.
- **Delete anomaly:** deleting a guest's only booking would delete the guest.
- **Repeating groups (1NF):** a stay has many service lines and many payments.

5.3 Normal forms applied

- **1NF:** all columns atomic; service lines and payments moved to their own tables; statuses are single-valued ENUMs.
- **2NF:** every table has a single-column surrogate primary key, so no partial dependency is possible.
- **3NF:** no non-key attribute depends on another non-key attribute. room stores type_id (not capacity); reservation stores rate_plan_id (not the rate); money collected lives in payment.

5.4 Functional dependencies

Table	Functional dependencies	Form
room_type	type_id → type_name, capacity, description; type_name → type_id	BCNF
room	room_id → room_number, type_id, status; room_number → room_id	BCNF
rate_plan	rate_plan_id → type_id, plan_name, nightly_rate, description	BCNF
guest	guest_id → full_name, email, phone; email → guest_id	BCNF
staff	staff_id → full_name, role	BCNF
reservation	reservation_id → guest_id, room_id, rate_plan_id, status, dates, adults, children, created_at	BCNF
stay	stay_id → reservation_id, actual_check_in, actual_check_out; reservation_id → stay_id	BCNF
housekeeping_task	task_id → all columns; stay_id → room_id (when present)	3NF exception †
service_request	service_request_id → stay_id, service_name, requested_at, status	BCNF
service_charge	charge_id → all columns; service_request_id → stay_id (when present)	3NF exception †

invoice	invoice_id → stay_id, invoice_date, subtotal, tax, total; stay_id → invoice_id	BCNF
payment	payment_id → invoice_id, payment_date, amount, method, status	BCNF

† Two deliberate exceptions: `service_charge.stay_id` and `housekeeping_task.room_id`. The parent link is optional (a charge may exist without a request, a task without a stay), so the value must be stored on the row. When the parent is present the value is derivable, so the redundancy is guarded by triggers that reject any row where the two disagree.

5.5 Design decisions

- `stay` is separate from `reservation` (1 : 0..1): actual times exist only if the guest arrives.
- `staff` is its own table so that names and roles are not repeated on every task.
- Invoice `subtotal`, `tax` and `total` are stored as a billing snapshot so a closed invoice does not change when rates change later.

6. Constraints, Triggers and Business Rules

Rules are enforced by the database, not only by the application, so no front end can bypass them.

#	Rule	Enforced by	Message on violation
1	No overlapping bookings for a room	Trigger	Room is already booked for an overlapping date range.
2	Occupants within room capacity	Trigger	Occupant count exceeds room type capacity.
3	Check-out after check-in	CHECK	chk_dates is violated.
4	Charge amounts not negative	CHECK	chk_charge_nonneg is violated.
5	Payments cannot exceed invoice total	Trigger	Payment would exceed the invoice total.
6	Rate plan belongs to the room’s type	Trigger	Rate plan does not belong to this room type.
7	Charge’s stay matches its request’s stay	Trigger	Service charge stay does not match its service request.
8	Task’s room matches its stay’s room	Trigger	Housekeeping task room does not match the stay’s room.

Two further triggers keep derived state tidy (invoice status follows payments; room and stay status follow check-in and check-out). Indexes support the availability lookup (`idx_reservation_dates`) and the common joins.

7. SQL Queries, Views and Reports

Query	Technique	Purpose
Q1	INNER JOIN, 4 tables	Every reservation with guest, room, type and rate
Q2	LEFT JOIN	Every room with its current guest (NULL when vacant)
Q3	JOIN with calculation	Invoice breakdown per stay
Q4	Nested subquery (NOT IN)	Rooms free for a requested date range
Q5	Subquery in HAVING	Stays with above-average service spend
Q6	GROUP BY	Reservations and average rate per room type
Q7	GROUP BY + HAVING	Housekeeping workload per staff member
Q8	Aggregate	Total revenue collected and service revenue
Q9	Subquery ratio	Cancellation rate
Q10–12	Views	Occupancy, guest history, housekeeping status
Q13	EXPLAIN	Availability lookup uses `idx_reservation_dates`

Views (8): v_current_occupancy, v_vacant_rooms, v_cancellations, v_housekeeping_status, v_guest_history, v_service_revenue, v_total_revenue, v_occupancy_rate.

```
-- [Q4] rooms free for a date range
SELECT r.room_number, rt.type_name
FROM room r JOIN room_type rt ON r.type_id = rt.type_id
WHERE r.status <> 'maintenance'
  AND r.room_id NOT IN (
      SELECT room_id FROM reservation
      WHERE reservation_status NOT IN ('cancelled','no_show')
        AND '2026-10-11' < planned_check_out
        AND '2026-10-12' > planned_check_in);
```

8. Application (Prototype)

The application layer is split so that business logic is independent of the interface. A shared `db.py` provides `run_query` and `run_transaction`; the latter runs several statements on one connection and commits or rolls back together, with `Ref(n)` placeholders to pass an auto-generated key from one statement to the next. All SQL is parameterized. Database errors are translated into plain messages (for example error 1062 becomes “That value must be unique and already exists”).

Front end	Technology	Notes
Command-line app (`app/`)	Python, mysql-connector	11 menu operations; first working prototype
Web UI (`app_web/`)	Python, Streamlit	Dashboard, booking, check-in and out, services, housekeeping, billing, reports
Web application (`app_site/`)	React, Vite, Flask API	“Taj Ritheek”: front desk, drag-and-drop reservation timeline, kanban housekeeping, bill preview with payment seal, command palette (Ctrl K), reports with charts. Automated end-to-end click-through test and API tests

The same site also runs in a **demo mode** with in-browser sample data, published on GitHub Pages, so it can be viewed without a database.

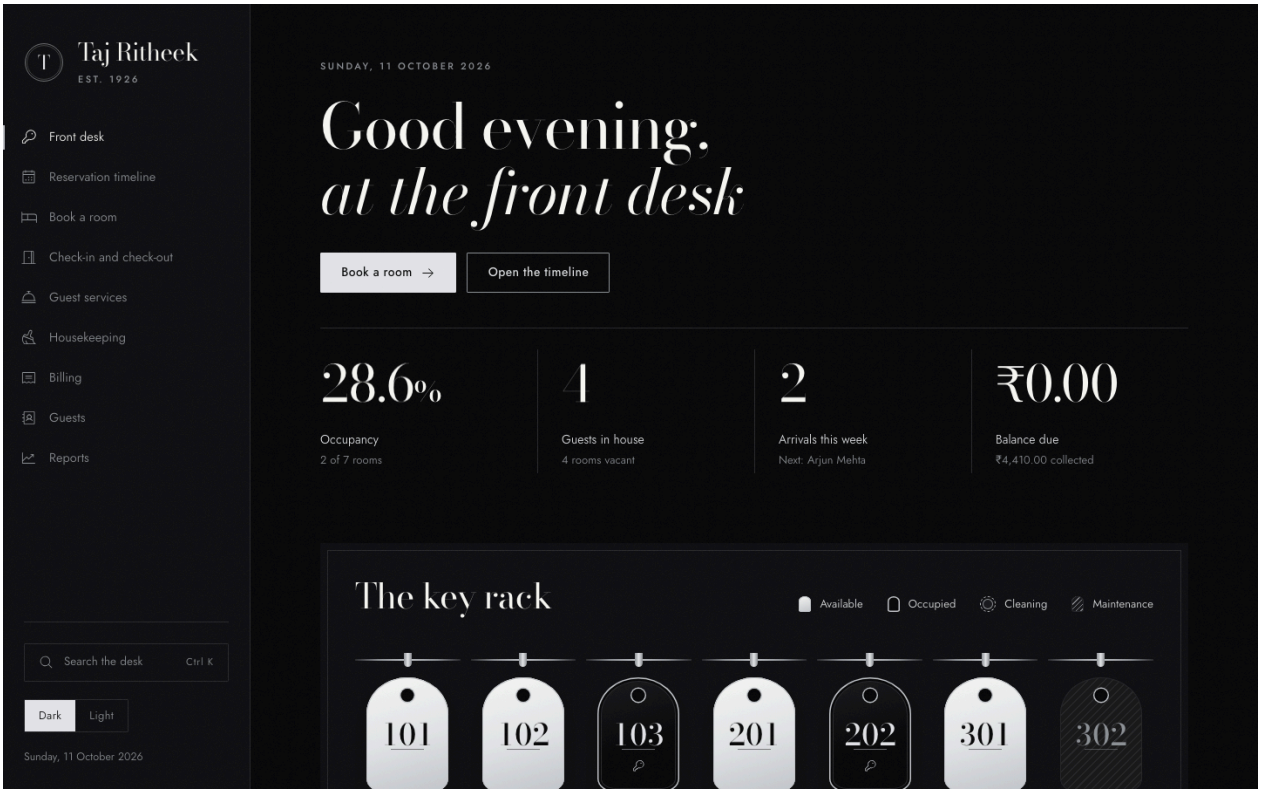


Figure 2. Front desk dashboard with occupancy, arrivals and the key rack.

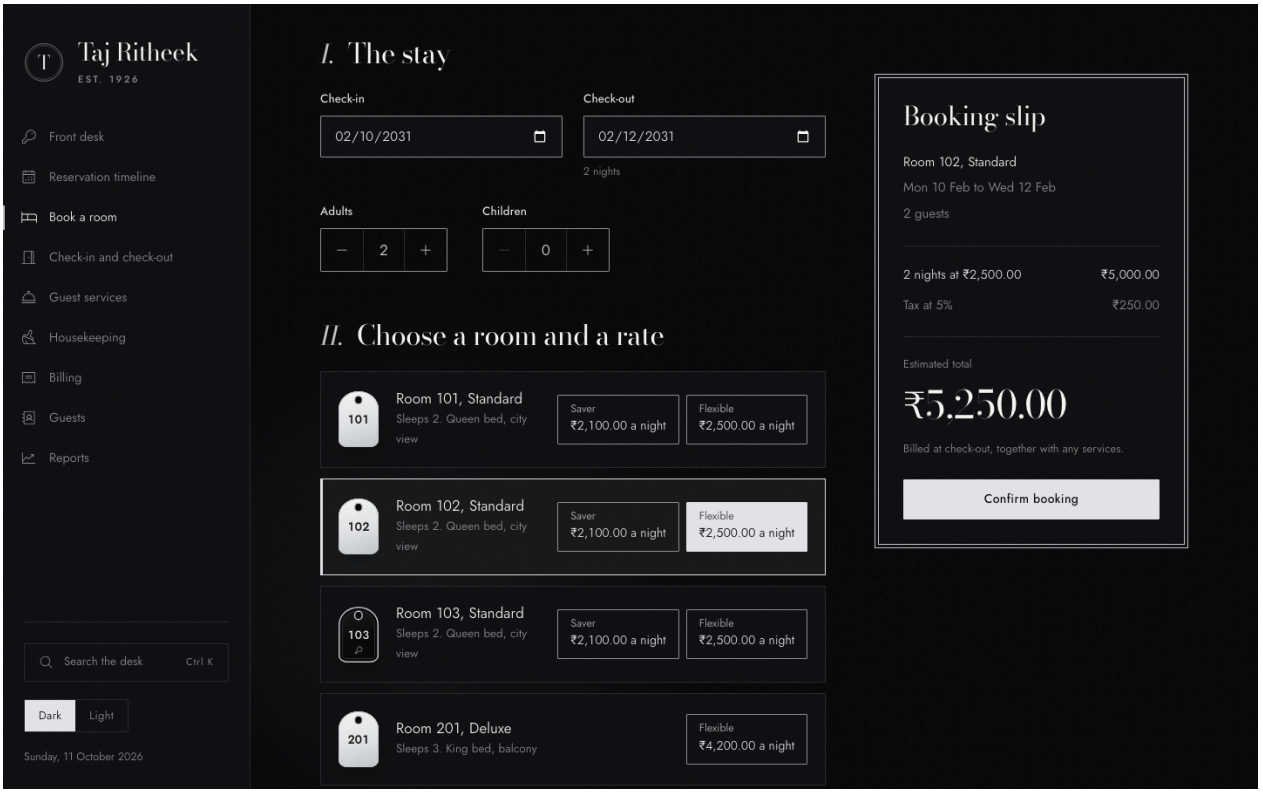


Figure 3. Booking a room: dates, room and rate; the booking slip prices the stay.

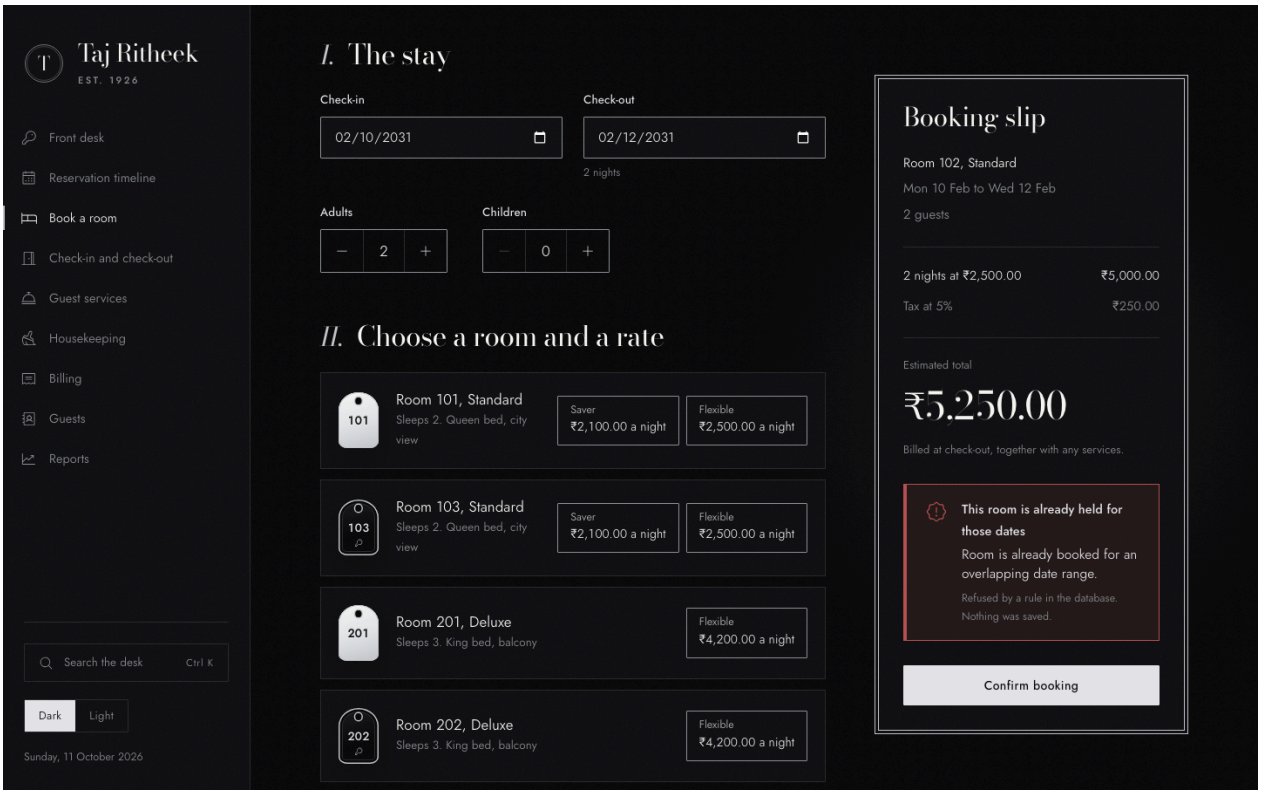


Figure 4. The database rejects an overlapping booking; the interface shows the rule's message and nothing is saved.

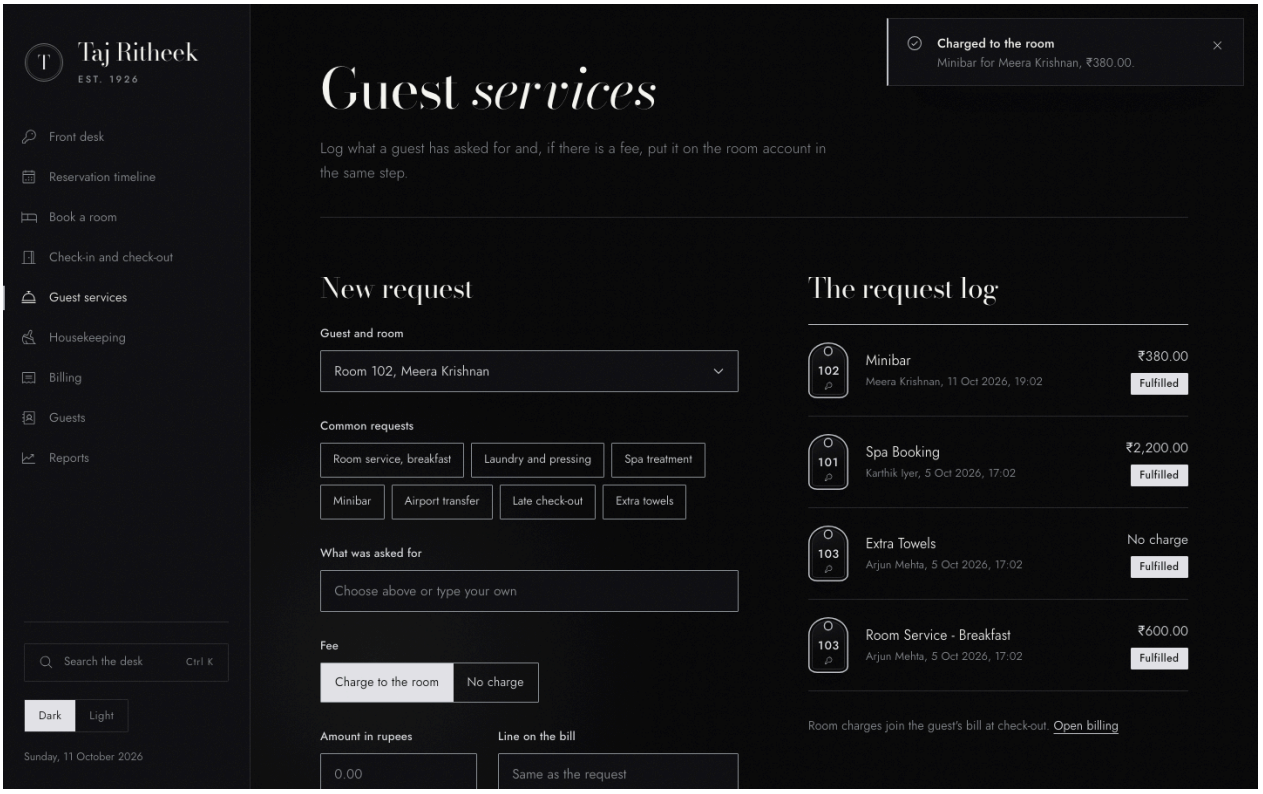


Figure 5. Posting an in-stay service charge.

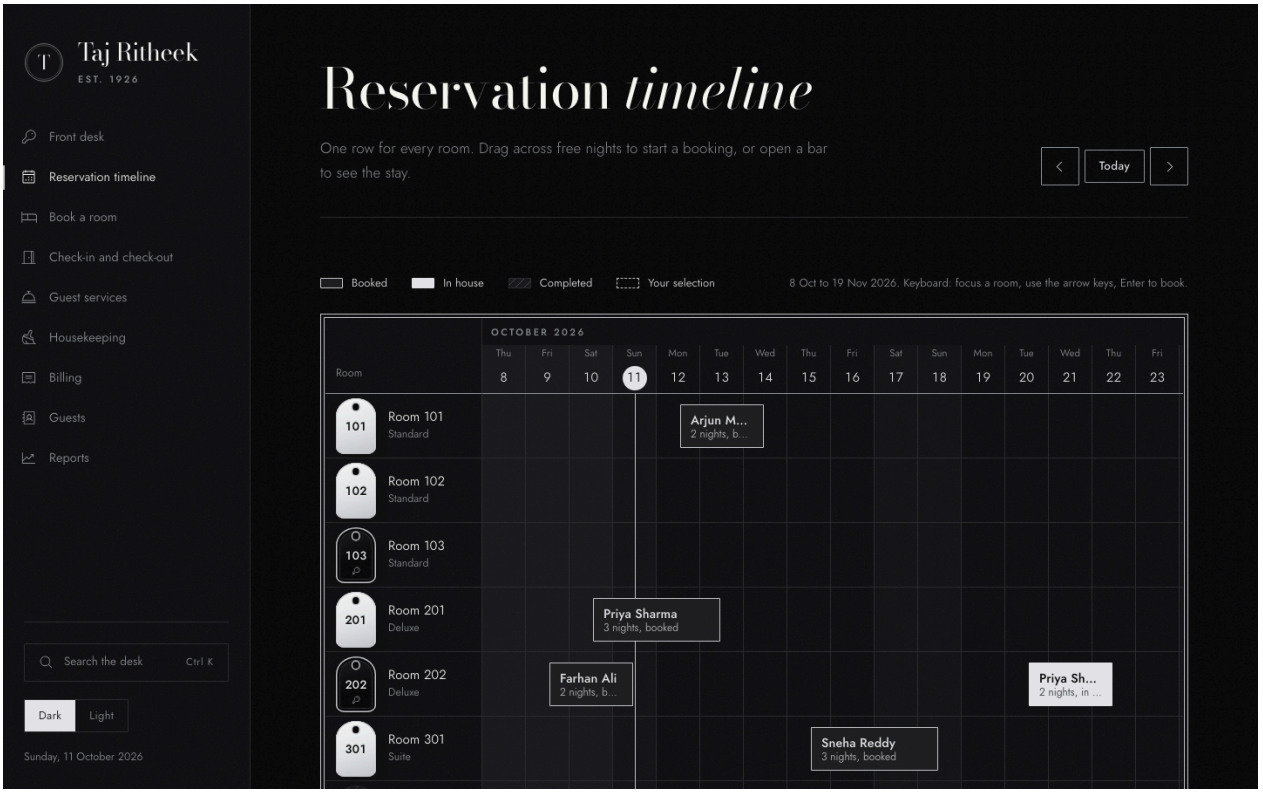


Figure 6. Reservation timeline.

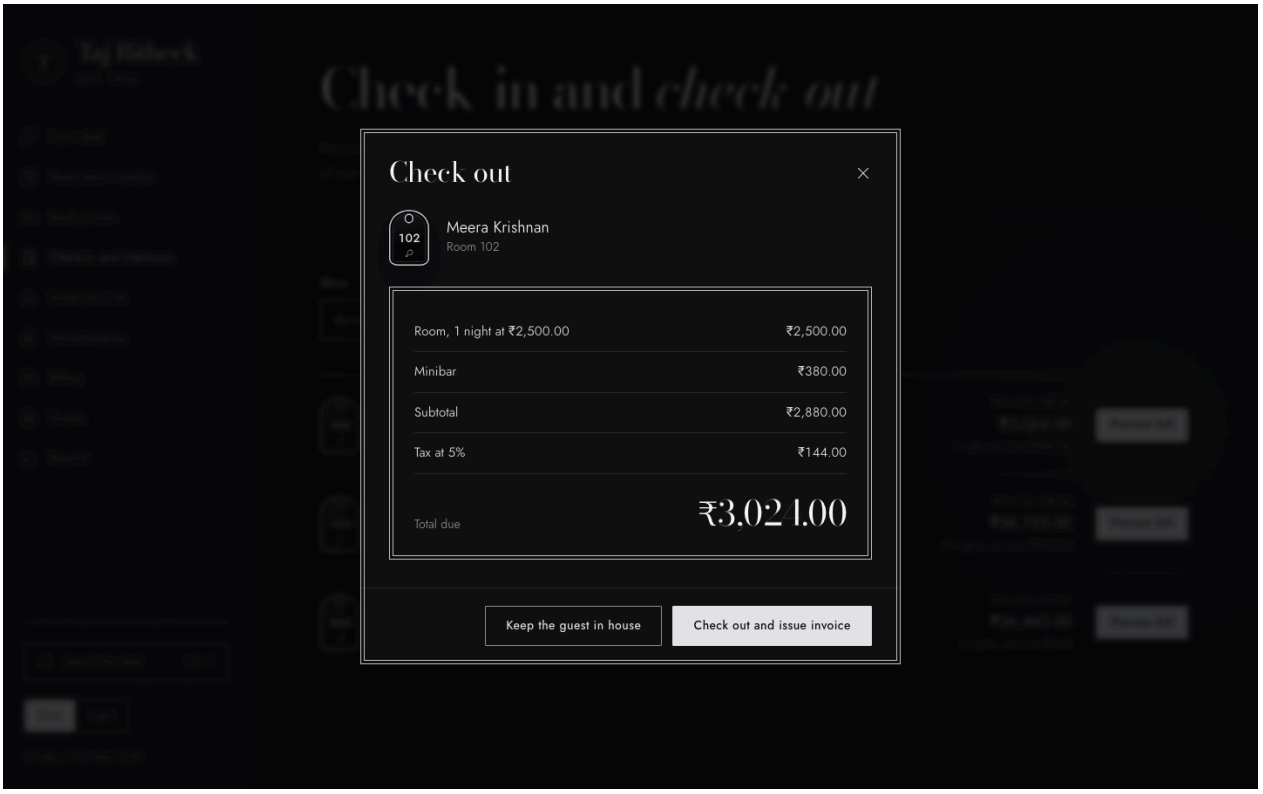


Figure 7. Bill preview at check-out: room charge, services, 5% tax and total.

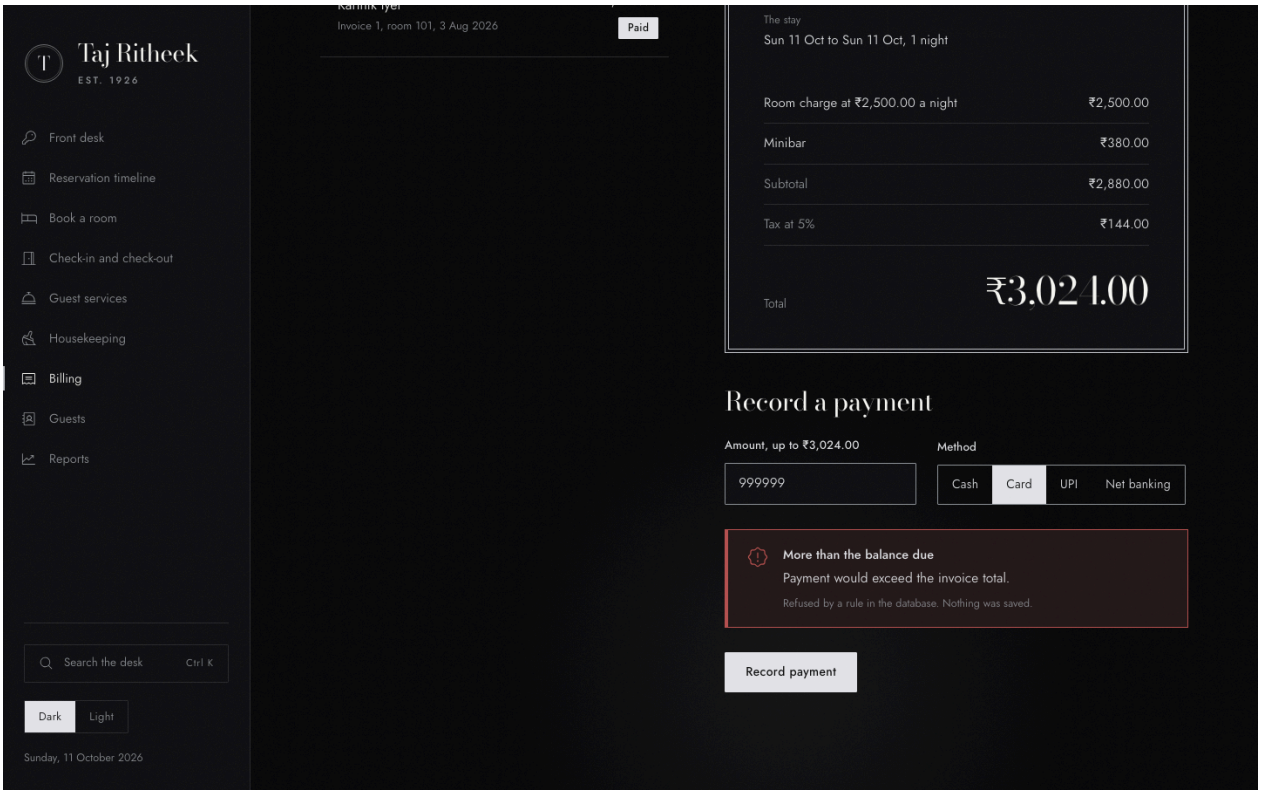


Figure 8. A payment larger than the invoice total is refused by the database.

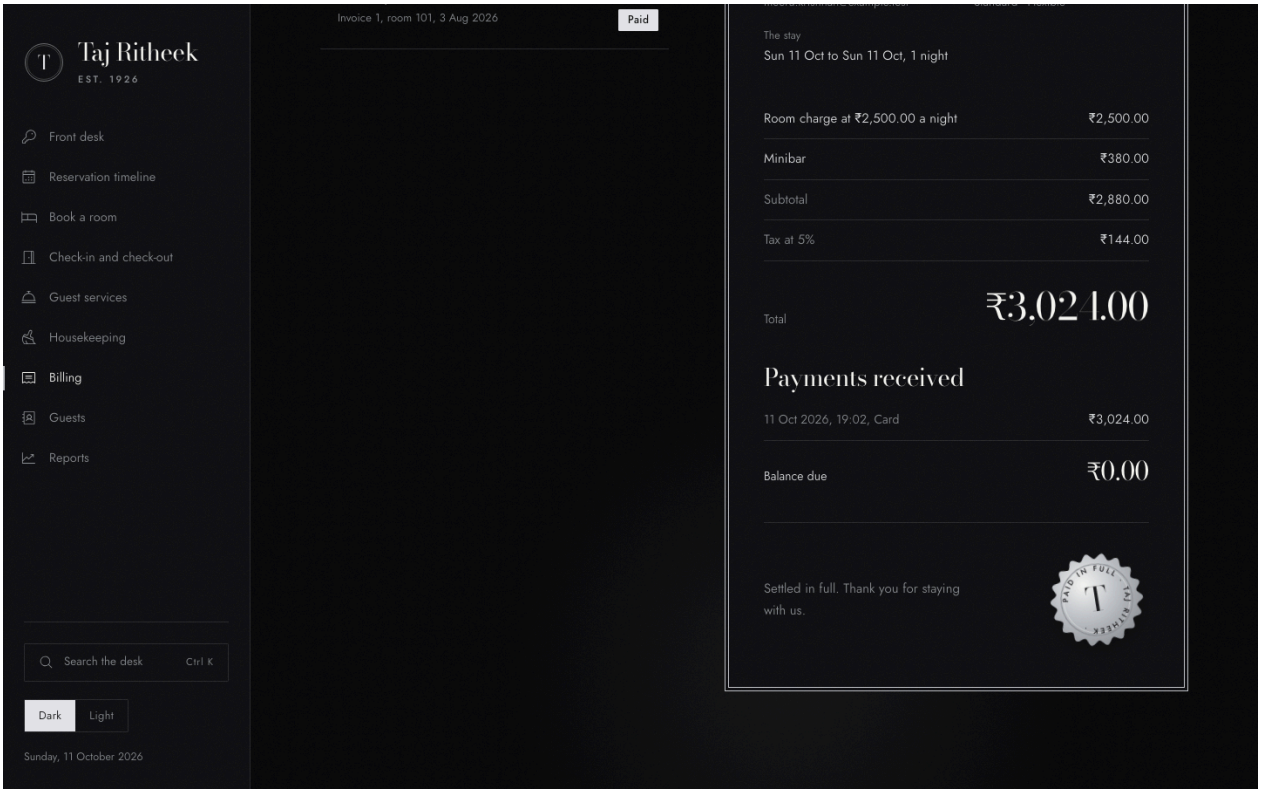


Figure 9. A fully paid invoice receives the paid seal.

```
mysql hotel_pbl (server 26.7.0)

mysql> SELECT invoice_id, total, status, (SELECT COUNT(*) FROM payment WHERE invoice_id = 1) AS payments FROM invoice WHERE invoice_id = 1;
+------------+---------+--------+----------+
| invoice_id | total   | status | payments |
+------------+---------+--------+----------+
| 1          | 4410.00 | paid   | 1        |
+------------+---------+--------+----------+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO payment (invoice_id, amount, payment_method) VALUES (1, 9999999, 'cash');
ERROR 1644 (45000): Payment would exceed the invoice total.

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT invoice_id, total, status, (SELECT COUNT(*) FROM payment WHERE invoice_id = 1) AS payments FROM invoice WHERE invoice_id = 1;
+------------+---------+--------+----------+
| invoice_id | total   | status | payments |
+------------+---------+--------+----------+
| 1          | 4410.00 | paid   | 1        |
+------------+---------+--------+----------+
1 row in set (0.00 sec)

-- trg_payment_limit_insert refused the payment; the transaction was rolled back and nothing changed.
```

Figure 10. Trigger rejection in the MySQL client: the over-payment fails with error 1644 and is rolled back.

9. Testing and Validation

Each business rule was deliberately violated against the live MySQL database. Every violation was rejected by a CHECK or trigger and the affected table’s row count was unchanged. A valid booking afterwards confirmed that only genuine violations are blocked.

Application-level test	Result shown to the operator
Overlapping date range	[Rejected] Room is already booked for an overlapping date range.
Impossible date typed	Invalid date format; prompt repeats
4 adults in a room for 2	[Rejected] Occupant count exceeds room type capacity.
Payment above invoice total	[Rejected] Payment would exceed the invoice total.
Duplicate guest email	[Rejected] unique value already exists
Delete a guest who has reservations	[Rejected] record still referenced
Cancel a checked-in reservation	Only booked reservations can be cancelled
Check out a closed stay	Stay not found or already checked out; original time unchanged

Transaction test: a two-statement transaction inserted a guest and then a second guest with a duplicate email. The error was raised and the first guest did not survive, so the rollback worked. A second transaction (guest, then a reservation using that guest’s new id) committed both rows.

Automated tests (web app): API tests plus an end-to-end click-through that books, rejects an overlap, checks in, posts a service, previews the bill, rejects an overpayment, and pays, with screenshots at each step.

10. Results, Limitations and Future Work

Results

- All nine functional requirements are implemented and exercised through three interfaces.
- All eight business rules are enforced in the database and verified by deliberate violations.
- Multi-step operations are atomic; no test left partial data.

Limitations

- Single property and a flat 5% tax rate.
- The hosted demo uses sample data in the browser, not a live database.
- Staff login is basic; there is no per-role permission model yet.

Future work

- Role-based access for front desk, housekeeping and accounts.
- Dynamic pricing by season, and multi-property support.
- Email confirmations and an online guest booking portal.

11. How to Run

```
mysql -u root -p < review-3/db/01_schema.sql      # then 02 to 04 in order
cp review-3/app/.env.example review-3/app/.env    # add DB credentials
pip install -r review-3/app/requirements.txt
python review-3/app/main.py                       # command-line app
streamlit run review-3/app_web/app.py             # Streamlit web UI
bash review-3/app_site/run.sh                     # React + Flask site
```

Full instructions are in `review-3/docs/` (how_to_run.md, how_to_run_web.md, how_to_run_site.md). Database credentials live in `.env`, which is never committed.

12. Conclusion

The project delivers a normalized, constraint-rich hotel database whose rules cannot be bypassed by any front end, and a working multi-interface prototype built on it. Placing integrity in the database kept the application code simple and made every rule testable on its own. The design extends cleanly to roles, pricing and online booking.

Deliverables and links

Item	Location
Review 1, 2, 3 decks (PDF)	review-1/, review-2/, review-3/ in the repository
Prototype source code	review-3/ in the repository
Live demo	https://ritheekyadav01-afk.github.io/taj-ritheek/
Video presentation (5 min)	video/VIDEO_LINK.md in the repository